

두 Event-action table의 구현과 Region 실행 구조 설명

2026-09-11 작업 폴더의 실제 소스 기준으로 작성했습니다. 설명을 위해 기존 코드를 읽고 관련 테스트를 실행했으며, 매매 구현은 수정하지 않았습니다. 아래 설명은 교수님께 프로젝트 구조를 발표하는 말투로 구성했습니다. 마지막에는 두 표의 **122개 ID 전체에 대한 소스 위치 색인**을 붙였습니다.

교수님, 이 프로젝트는 **4시간봉으로 시장 유형을 추천하는 STM**과 선택된 유형에 따라 **진입·보유·청산**을 판단하는 **TradingSTM**을 분리했습니다. 두 STM 모두 “무엇을 해야 하는가”를 결정하고, Controller가 실제 데이터 변경과 주문을 수행하도록 구성했습니다. TradingSTM 안에서는 포지션 관리, Case B 신호, Case C 신호를 세 Region으로 나누었습니다. 세 영역은 상태를 동시에 유지하면서 같은 이벤트에 반응하지만, 실제 판단과 실행 순서는 하나의 직렬 처리 흐름으로 제어합니다.

1. 설계 문서와 구현 폴더가 어떻게 대응하는가

설계 자료	핵심 판단 코드	실제 Action 수행 코드
4H_REGIME_Event_Action_Table.md	stm.py와 transitions.py	regime_controller.py
Trading_Logic_Event_Action_Table.md	stm.py와 아래의 여섯 전이 파일	trading_controller.py

설계 문서가 있는 폴더는 설명과 명세를 보관합니다. 실제 프로그램은 `backend/src/binance_auto_trader` 아래에 있습니다. `domain`에는 상태·이벤트·조건·전이와 Action 요청의 자료형을, `application`에는 입력 준비와 Action 실행을, `adapters`에는 거래소 통신과 저장을 둡니다. `bootstrap`은 이 객체들을 연결하고 실행 작업을 구동합니다.

표의 한 행은 코드에서 대체로 다음처럼 읽을 수 있습니다.

```

현재 상태 + 입력 이벤트 + Guard
→ transition ID와 다음 상태 결정
→ 실행할 Action 요청들을 순서대로 반환
→ Controller가 Context·주문·저장 작업 실행
→ 결과 이벤트를 다음 판단으로 전달
  
```

이 구분 때문에 `stm.py`만 읽으면 주문이나 타이머 구현이 보이지 않습니다. 표의 한 행 전체를 설명하려면 **전이 파일** → **Action 자료형** → **Controller 실행 함수**까지 따라가야 합니다.

2. 첫 번째 표: 4H REGIME 추천 STM

교수님, 4H 추천부는 세 개의 병렬 Region을 가진 구조가 아닙니다. `INITIAL`, `FOUR_HOUR_CANDLE_EVALUATION`, 다섯 추천 상태 중 **현재 상태 하나**를 갖는 별도 상태 머신입니다. 상태 이름은 `states.py`, 이벤트 이름은 `events.py`에 정의했습니다.

`transitions.py`의 `RegimeTransition` 목록에는 표의 13개 행을 실제 데이터로 등록했습니다. 각 항목은 ID, 시작 상태, 이벤트, Guard, 다음 상태와 Action 생성에 필요한 정보를 가지고 있습니다. `handle`은 현재 상태와 이벤트의 전이 후보를 조회하고 Guard가 맞는 전이를 선택한 뒤, 자신의 상태를 변경하고 `RegimeSTMResult`를 반환합니다.

표 ID	구현한 판단과 요청
EA-001	최초 평가 요청을 받으면 평가 상태로 들어가 <code>StartRegimeEvaluation(INITIAL)</code> 을 반환합니다.
EA-002	EMA9 기울기가 -0.15 이상 0.15 이하이면 TYPE_0 적용을 요청합니다.
EA-003	0.15 초과 0.30 미만이면 TYPE_1 적용을 요청합니다.
EA-004	0.30 이상이지만 강상승 구조와 가격 위치의 복합조건이 충족되지 않으면 TYPE_1을 요청합니다.
EA-005	0.30 이상이고 HH·HL이 모두 참이며 현재가가 live EMA9 이상이면 TYPE_2를 요청합니다.
EA-006	-0.30 초과 -0.15 미만이면 TYPE_3을 요청합니다.
EA-007	-0.30 이하이지만 강하락 복합조건이 충족되지 않으면 TYPE_3을 요청합니다.
EA-008	-0.30 이하이고 LH·LL이 모두 참이며 현재가가 live EMA9 이하이면 TYPE_4를 요청합니다.
EA-101~105	각 추천 상태에서 새 4H 봉이 마감되면 평가 상태로 돌아가 재평가 시작을 요청합니다.

이 수치 조건은 `guards.py`의 일곱 함수에 있습니다. 입력은 `evaluation.py`의 `RegimeEvaluationContext`로 고정하고, 요청과 결과의 형식은 각각 `action_requests.py`, `results.py`에 둡니다.

실제 지표 계산은 `calculate_4h_indicators`가 담당합니다. 같은 시장 snapshot을 바탕으로 EMA9, 기울기, 고점·저점 구조, live EMA9를 준비합니다. 입력이 부족하거나 유효하지 않으면 정상적인 추천 판단을 시작하지 않습니다.

이후 `_run_prepared_evaluation`에서 한 평가를 두 단계로 진행합니다.

1. 준비한 입력에 `evaluation_id`를 부여하고 최초 평가 또는 4H 마감 이벤트를 STM에 보냅니다.
2. STM의 `StartRegimeEvaluation` 요청을 검증합니다.
3. 같은 ID를 가진 `EVALUATION_READY`와 평가 Context를 STM에 보냅니다.
4. STM이 반환한 `ApplyRecommendedRegime`을 검증하고 `_apply_recommended_regime`에서 추천값과 결과를 적용합니다.

예를 들어 최초 입력이 기울기 0.35, HH·HL 참, 현재가 $\geq$ live EMA9이면 `EA-001` → `EA-005`를 거쳐 TYPE_2를 추천합니다. 그 다음 4H 마감에서는 현재 TYPE_2 추천 상태의 `EA-103`을 거친 뒤 새로운 입력에 따라 EA-002~008 중 하나를 선택합니다.

중복된 4H 봉, 과거 봉, 서로 다른 시장 version이 섞인 입력은 Controller에서 검사합니다. 시장 데이터와의 연결은 `market_data_controller.py`에 있으며, `_commit_atomic_boundary`는 4H 마감이 포함된 봉 경계의 데이터를 같은 version으로 반영한 뒤 추천 평가를 호출합니다.

여기서 추천은 사용자 선택과 구분했습니다. 추천이 TYPE_2가 되었다고 실행 중인 거래 전략이 자동으로 TYPE_2로 교체되는 구조는 아닙니다. 실제 선택은 `set_regime_type`와 TradingController의 선택 처리 경로를 거칩니다. 더구나 현재 `logic_registry.py`는 매매 구현을 TYPE_0에만 연결하고 TYPE_1~4는 `UNSUPPORTED_TRADING_LOGIC`으로 차단합니다. 추천 다섯 종류의 구현과 매매 다섯 종류의 구현은 범위가 다릅니다.

3. 두 번째 표: TradingSTM의 109개 전이는 어디에 있는가

교수님, TradingSTM은 하나의 거대한 조건문에 모든 전략을 넣지 않고, 표의 책임 구분에 따라 아래처럼 나누었습니다. 각 파일의 `handle..._transition()`이 해당 영역의 상태·이벤트를 검사하고 전이 결과를 만듭니다.

표 ID 묶음	파일	담당하는 판단
G-01~07 및 G-06P/F/R	global_transitions.py	시작, 하단 터치, 새 터치로 재초기화, 완료, 중지, 상단 BB 안전 종료
O-01~09	ownership_transitions.py	체결 후 소유권 확정, B/C 매수 실패와 재시도
PB-01~24 및 PB-23F	case_b_position_transitions.py	B 보유, 손절·익절·시간 청산, TREND_HOLD, 매도 실패·완료
PC-01~28 및 PC-23F	case_c_position_transitions.py	C 보유, TP_TRAILING, 청산, 회복 확인과 B 인계
B-01~19	case_b_signal_transitions.py	터치봉 BBW, 최초 신호봉, 놀림목 매수, C 보유 중 진입 일시정지
C-01~17	case_c_signal_transitions.py	setup, flush 저점, 3분 회복, 매수 또는 진입 종료

초기 진입 행에는 예외가 있습니다. **O-01·B-01·C-01**은 **G-02/G-03** 등의 복합 상태 진입에서 함께 기록하고, **PB-01·PC-01**은 **O-02/O-06**에서 포지션 하위 상태를 열 때 함께 기록합니다. 따라서 해당 ID가 같은 이름의 전이 파일 안에만 있을 것이라고 찾으면 놓칠 수 있습니다. 부록은 이 실제 위치를 반영했습니다.

[catalog.py](#)에는 109개 ID 전체가 등록되어 있습니다. [_commit](#)은 결과의 ID가 등록된 ID인지 확인합니다. 다만 ID 목록에 존재하는 것만으로 모든 Guard와 실행 효과가 완전히 검증되었다고 볼 수는 없습니다. 실제 분기와 테스트를 함께 확인해야 합니다.

4. 세 Region은 코드에서 어떻게 나뉘는가

핵심은 [TradingStateConfiguration](#)입니다. 하나의 불변 객체에 다음 상태들을 별도 필드로 저장합니다.

```
TradingStateConfiguration
  root_state
  ownership_state      ← Region 1
    case_b_position_state ← B 소유권의 하위 상태
    case_c_position_state ← C 소유권의 하위 상태
  case_b_signal_state   ← Region 2
  case_c_signal_state   ← Region 3
```

B/C 포지션 필드가 따로 있어도 독립적인 Region이 다섯 개라는 뜻은 아닙니다. **B와 C의 포지션 관리는 Region 1 안에서 둘 중 하나만 활성화되는 하위 상태**입니다. 상태 객체의 검증 코드가 B/C 포지션 하위 상태를 동시에 켜는 구성을 거부합니다.

[create_trade_management_initial_state](#)는 하단 터치 후 아래 상태들을 한꺼번에 만듭니다.

영역	최초 활성 상태	이후 독립적으로 기억하는 내용
Region 1	NO_POSITION	누가 포지션을 관리하는지, 보유·청산·회복 단계
Region 2	B_WAIT_TOUCH	B가 신호봉을 기다리는지, 놀림목을 기다리는지, 진입 결과를 기다리는지
Region 3	C_WAIT_SETUP	C가 setup을 기다리는지, flush·회복을 관찰하는지, 진입 결과를 기다리는지

세 Region은 각자의 상태를 유지하지만 입력과 거래 자원은 공유합니다. `context.py`에는 시장 평가, 실제 Position, runtime 변수와 읽기 전용 `TradingContextView`가 있습니다. `signal_time`, `flush_low`, `timer_base_time`, pause flag, pending 주문과 소유자는 여기에서 관리합니다. 상태 필드는 STM이 바꾸고, runtime 값은 Controller가 Action을 통해 바꿉니다.

또한 `ownership_state`는 상태 머신의 관리 단계이고 `runtime.position_owner`는 실제 체결이 반영된 포지션 소유자입니다. C가 전량 매도된 뒤에도 회복을 기다리는 동안 Region 1은 `CASE_C_CLOSED`를 유지할 수 있습니다. 이때 실제 포지션 소유자는 이미 `None`입니다. 두 값은 이 과정에서 같은 의미가 아닙니다.

5. '병렬' 의도는 어떤 실행 방식으로 구현했는가

교수님, 현재 구현은 여러 Region의 상태를 함께 유지하고 이벤트를 여러 영역에 전달하는 논리적 동시성을 구현했습니다. Region마다 별도 스레드를 배정하는 물리적 병렬 실행은 구현하지 않았습니다.

실제 실행 순서는 `_handle_locked`에서 확인할 수 있습니다.

1. 전역 전이를 먼저 확인합니다. STOP, 상단 안전 종료, 복합 상태 재진입 같은 전역 전이가 선택되면 그 결과를 확정합니다.
2. Region 1을 평가합니다. B 포지션이 활성화되어 있으면 B 포지션 함수, C이면 C 포지션 함수, 둘 다 아니면 소유권 함수를 호출합니다.
3. Region 3, 즉 C 신호를 평가합니다.
4. Region 2, 즉 B 신호를 평가합니다.
5. 선택한 전이들의 상태와 Action 요청을 모아 한 결과로 확정합니다.

이 순서는 코드에 명시되어 있습니다. 세 함수가 서로 다른 스레드에서 동시에 실행되는 구조가 아닙니다. 같은 microstep에서는 모든 영역이 같은 불변 Context를 읽습니다. 다만 `state_after`는 앞 영역의 전이가 반영된 상태를 다음 영역으로 전달합니다. 그러므로 각 영역이 완전히 격리된 초기 상태만 보고 투표하는 모델도 아닙니다. 순서가 정해진 상태 합성과 공유 입력을 사용하는 방식입니다.

여기서 microstep은 이벤트 하나를 판단하고 그 Action 묶음을 처리하는 단위입니다. `handle`의 비재진입 Lock은 두 호출이 동시에 같은 STM을 변경하는 것을 거부합니다. `RunToCompletionEventProcessor`는 이벤트 하나의 Action 묶음을 끝낸 후 다음 이벤트를 처리합니다.

`SerialEventQueue`는 우선순위와 FIFO 순서를 사용합니다. `QueueEvent`로 만든 내부 후속 이벤트와 Controller의 주문 결과 이벤트는 내부 우선순위로 넣어, 이미 대기 중인 시장 이벤트보다 먼저 처리합니다. STM을 재귀 호출해서 즉시 다른 Region을 실행하는 방식은 쓰지 않습니다.

실제 백그라운드 실행은 `_TradingEventRuntimeWorker`가 담당합니다. 이 작업자는 하나의 스레드에서 Controller의 `run_event_runtime_cycle()`를 구동합니다. 기본 확인 주기는 0.25초이고 새 처리 요청으로 깨울 수도 있습니다. `asyncio.run()`으로 비동기 형태의 처리 함수를 실행하더라도 내부 Region 처리는 앞서 설명한 순차 호출입니다. 이 주기는 전략이 0.25초마다 새 주문을 내거나 Region마다 타이머 스레드를 만든다는 뜻이 아닙니다.

이 구조는 동시에 진행되는 B/C 감시를 표현하면서 주문 충돌을 통제하고 동일 입력의 결과를 재현하기에 적합합니다. 대신 긴 Action 실행은 후속 이벤트 처리를 지연시킬 수 있으며, Region별 CPU 병렬 처리 성능을 얻는 구조는 아닙니다. 의도한 병렬성이 "B 신호를 기억하면서 C를 관리한다"라면 현재 구현에 반영되어 있습니다. "각 Region을 별도 스레드·태스크에서 동시에 실행한다"는 의미였다면 그 부분은 현재 구현되어 있지 않습니다.

6. 같은 순간에 B와 C가 모두 매수하려 하면 어떻게 하는가

`_does_case_c_buy_win`은 앞서 채택된 C의 `SubmitOrder(BUY)`와 B의 매수 후보를 비교합니다. 두 후보가 같은 microstep에 함께 생겼으면 B 후보의 상태 전이와 Action 묶음을 채택하지 않고 C를 유지합니다. B의 상태가 매수 요청 완

료로 잘못 넘어가지 않도록 후보 전체를 제외합니다.

예를 들어 B는 이미 **B_WAIT_PULLBACK**, C는 **C_SETUP**인 상태에서 **%B=-0.18**, B 신호 경과 1시간, C 회복 경과 2분, C 진입선 -0.18이라고 하겠습니다. 나머지 허용 조건도 맞으면 B와 C가 모두 매수 후보가 될 수 있습니다. 이 경우 C-12만 채택되어 C 주문 하나를 요청하고 B는 놀림목 대기를 유지합니다. 이 예시는 기존 테스트의 실제 검증 입력과 같습니다.

그 다음 **create_entry_order_actions**가 만든 요청에 따라 Controller는 **pending_strategy**, 주문 방향·의도 ID와 **ENTRY_ORDER_PENDING**을 적용합니다. 주문 진행 중에는 pending 주문과 실행 단계가 새로운 주문의 진입을 통제합니다. 결과가 불명확하면 같은 주문을 조회·조정합니다.

C 우선은 같은 **microstep**의 동시 매수 후보에 대한 규칙입니다. 이미 앞선 이벤트에서 B 주문이 제출된 후까지 C가 무조건 선점한다는 뜻으로 확대하면 안 됩니다. 이벤트 간에는 큐 순서와 pending 주문, 취소·조회 정책이 작동합니다.

7. C 보유 중에도 B 감시를 유지하는 실제 예시

교수님, 이 부분이 Region을 나눈 가장 직접적인 효과입니다. C 매수 체결을 Position·이력·Context에 반영한 뒤 Controller가 **CASE_C_POSITION_OPENED**를 큐에 넣습니다. B가 놀림목 대기 중이라면 이벤트 하나가 다음 네 ID를 함께 발생시킵니다.

ID	처리 영역	결과
O-06	Region 1	C 포지션 관리로 들어갑니다.
PC-01	Region 1의 C 하위 상태	CASE_C_HOLDING 을 시작합니다.
C-16	Region 3	C 신호 검사를 Final로 보냅니다. 신호의 역할은 끝났기 때문입니다.
B-13	Region 2	B_WAIT_PULLBACK 을 유지하고 case_b_entry_paused=True 를 요청합니다.

이때 전체 상태는 아래처럼 됩니다.

```

root_state           = TRADE_MANAGEMENT
ownership_state       = CASE_C_POSITION_MANAGEMENT
case_c_position_state = CASE_C_HOLDING
case_b_signal_state   = B_WAIT_PULLBACK
case_c_signal_state   = CASE_C_FINAL_STATE
  
```

C의 보유·청산 판단은 Region 1에서 계속되고, B는 신호 시각을 잃지 않은 채 Region 2에서 대기합니다. B의 3시간 타이머를 멈추거나 0으로 돌리지 않습니다. 시장 갱신이 들어오면 B는 경과 시간을 다시 확인하고, 3시간을 초과하면 B-11로 신호를 폐기할 수 있습니다. B가 아직 **B_WAIT_SIGNAL**이라면 B-12로 진입만 잠그며 확정 30분봉 신호 감시를 유지할 수 있습니다.

일반 **MARKET_DATA_UPDATED**를 각 영역에 맞는 조건 검사 이벤트로 바꾸는 **_resolve_region_one_event**, **_resolve_case_c_region_event**, **_resolve_case_b_region_event**도 TradingSTM 안에 있습니다. 따라서 시장 갱신 하나가 C 보유 판단과 B 감시의 재평가가 계기가 될 수 있습니다.

C 청산 이후에는 PC-24로 B 진입을 잠근 채 회복을 기다리고, **%B >= 0.25**가 되면 PC-26을 거칩니다. 청산 사유가 **TP_TRAIL**이고 청산 시 **%B < 0.40**이면 PC-27이 **CASE_B_ACTIVE_RESUME**를 생성하여 B-14/B-15에서 진입을

다시 허용합니다. 그렇지 않으면 PC-28이 **CASE_B_WAIT_ONLY**를 생성하여 B-16/B-17에서 매수 금지를 유지합니다. 이 미 Final에 도달한 B 신호를 이 이벤트가 자동으로 새로 만드는 것은 아닙니다.

즉 C 청산부가 B 상태를 직접 호출해 강제로 바꾸는 대신, **공유 flag를 적용하고 의미 있는 인계 이벤트를 전달하도록 연결했습니다.**

8. Case별 조건과 청산 로직은 어떻게 구현했는가

Case B 신호부는 터치 당시의 BBW가 0.02 미만인 경우 B-03으로 감시를 시작합니다. B-05는 확정 30분봉에서 기울기 > -0.03 , $\%B > 0.25$, 저가 $\geq$ 직전 확정 3개 봉의 최저 저가를 확인하고 최초 신호봉과 시각을 저장합니다. B-06/B-09는 신호 후 3시간 이내, $\%B \leq 0.30$, 무포지션·주문 없음·일시정지 해제 조건을 확인하여 매수를 요청합니다.

Case C 신호부는 $\%B \leq -0.15$ 와 $CCI \leq -140$ 일 때 setup을 열고, $\%B \leq -0.25$ 에서 첫 flush를 기록합니다. 더 낮은 가격 이 생기면 저점과 시간 기준을 갱신합니다. 진입선은 기준 $\%B + 0.06$ 이며, 3분 이내 회복하고 진입선 자체가 -0.15 미만일 때 C-12 매수를 요청합니다. 3분을 넘으면 C-11로 기준을 다시 잡습니다. 코드의 분기 순서상 더 낮은 저점 갱신 C-10이 시간 재시작 C-11보다 우선합니다.

이 비교에 쓰는 공통 조건과 수치 기준은 [conditions.py](#)에 있습니다. 예를 들어 **b_signal_age**는 10,800초 이하, **c_recovery_window**는 180초 이하입니다. 각 전이 함수는 **condition_met()** 또는 **condition_unmet()**을 호출합니다. 따라서 전이 파일에서 수치가 직접 보이지 않으면 이 조건 정의까지 읽어야 합니다.

보유 상태	판단 순서 및 코드
B CASE_B_HOLDING	_select_holding_exit_event : 긴급 손절 → 확정봉 손절 → TREND_HOLD 진입 → 익절 → 시간 청산
B CASE_B_TREND_HOLD	_handle_trend_hold() 에서 EMA 기울기 또는 %B 약화에 따른 매도와 재시도 처리
C CASE_C_HOLDING	_select_holding_event : 익절권 진입 → 손절 → 시간 청산
C CASE_C_TP_TRAILING	_select_trailing_event : $\%B < 0.10$ → 확정 1분봉 시점의 slope 비교 → 시간 청산

C의 trailing 비교에 쓰는 값은 **1분봉 EMA의 기울기 자체가 아닙니다**. [ThirtyMinuteMarketEvaluationBuilder](#)에서 확정 1분봉 close를 진행 30분봉의 후보 close로 사용해 30분 EMA slope를 계산합니다. TP 기준가격에 대한 slope도 미리 준비합니다. PC-10은 그 기준값을 **previous_trail_ema_slope**로 저장하고, PC-15는 slope 증가를 확인했을 때 비교 기준을 갱신합니다. TP_TRAILING 상태에서는 보유 초기의 C 손절 조건을 다시 적용하지 않습니다.

연속 5초·3분 같은 유지 조건은 STM 내부의 대기나 sleep으로 구현하지 않았습니다. [market_condition_timers.py](#) 및 [trading_indicator_timers.py](#)와 시장 평가 builder가 유지 상태를 계산하고, STM은 전달된 평가값을 읽습니다. B의 3시간과 C의 회복 경과 시간은 [_enrich_market_evaluation_elapsed](#)에서도 기준 시각으로부터 계산합니다.

표에 'RETRY 이벤트 재발생'이라고 적힌 대기 행은 주로 **ScheduleReevaluation**으로 구현했습니다. [_EventDrivenScheduler](#)가 시장 변화, 봉 마감, 기한 또는 재시도 backoff에 따라 예약을 해제합니다. 반면 다음 판단을 즉시 이어야 하는 인계·초기 검사 등은 **QueueEvent**를 사용합니다. 이 차이로 의미 있는 입력 변화 없이 무한 재평가하는 것을 피합니다.

9. Action 열은 실제로 어디까지 실행되는가

Action의 자료형은 [action_requests.py](#)에 있고, 분배 지점은 [_execute_action](#)입니다.

STM의 요청	Controller의 실제 수행
PatchRuntimeContext	TradingContext.apply_runtime_patch()로 runtime 값 변경
OpenLowerEvent / CloseLowerEvent	터치 snapshot과 하단 이벤트 범위 생성·종료
ResetCaseBContext / ResetCaseCContext	Case별 신호·setup 정보 정리
ScheduleReevaluation / CancelScheduledEvaluation	scheduler 등록·취소
QueueEvent	실행 요청을 기록하고, event processor가 전체 Action 묶음 이후 큐에 삽입
SubmitOrder	_submit_order_action()으로 주문 의도, 수량·위험 검사, 제출과 결과 조정
CancelPendingOrder / ReconcileOrder	pending 주문 취소, 동일 ID 조회, 실제 체결 반영
ForceSellAll	중지·안전 종료의 잔여 포지션 청산 조정
StopTradingRuntime	세션 타이머·구독 등 실행 자원 정리

주문 요청은 `_submit_order_action`에서 실행 경로로 이어지고, 거래소 경계는 `api_gateway.py`와 `spot_rest_client.py`가 담당합니다. 실제 Position 처리는 `position.py`, 이력 조정은 `trade_history_controller.py`, 저장은 `trade_history_repository.py`에 연결됩니다.

Controller의 `_handle_order_result`, `_apply_unapplied_fills()`, `_finalize_terminal_execution()`, `_complete_terminal_after_history()`와 `_create_order_outcome_event()`가 제출 이후 결과를 조정합니다. 정상 체결을 실제 Position과 저장에 반영한 다음 STM에 결과 이벤트를 전달합니다. 주문을 요청했다는 이유만으로 보유 또는 청산 완료 상태를 확정하지 않습니다.

예를 들어 B 매수는 B-06/B-09 → SubmitOrder → 실제 체결·이력 반영 → CASE_B_POSITION_OPENED → 0-02/PB-01/B-18로 이어집니다. C 신호가 활성 상태이면 같은 체결 이벤트에 C-06/C-15/C-17도 반응할 수 있습니다. B 매도는 해당 매도 요청 전이 이후 CASE_B_SELL_FILLED → PB-23F → CASE_B_SELL_FINISHED → PB-23 또는 PB-24를 거칩니다.

매도 요청 중에는 `pending_exit_reason`과 복귀 상태를 보존하며, 일반 보유 조건 검사보다 주문 결과·재시도를 처리합니다. 부분 체결이나 결과 불명 상황에서는 같은 주문 ID를 조회·조정합니다. 재시도에는 같은 청산 의도 키를 유지합니다. 이는 여러 Region이 동일 포지션을 대상으로 중복 주문을 만들어 내지 않도록 하는 실행부의 장치입니다.

실제 외부 주문 효과는 `_order_pipeline_enabled` 등 실행 조건도 통과해야 합니다. 따라서 STM이 SubmitOrder를 반환했다는 사실만으로 실제 거래소 주문까지 발생했다고 단정할 수는 없습니다. 실행 경로를 비활성화한 테스트 구성에서는 요청을 보존하고 결과 이벤트를 수동 공급할 수 있습니다.

10. Region 전체의 완료와 중지는 어떻게 합치는가

`trade_management_is_complete`는 Region 10이 NO_POSITION이고 B·C 신호 Region이 모두 Final인지 확인합니다. 마지막 미완료 영역까지 완료되면 TradingSTM이 TRADE_MANAGEMENT_COMPLETED를 요청하고, G-04에서 하단 터치 감시로 돌아갑니다. C 신호가 먼저 Final이 되었다는 이유만으로 C 보유 관리까지 끝내지 않습니다.

새로운 하단 터치 G-03은 허용 조건을 확인한 뒤 기존 이벤트를 닫고 세 Region을 초기 구성으로 다시 엽니다. 이전 터치 범위의 낮은 재평가 이벤트는 **lower_event_id** 검사로 배제합니다. B 손절 후 즉시 재진입할 수 있는 PB-23도 세 영역의 초기 상태를 다시 만들고 활성화 이벤트를 보냅니다.

STOP과 상단 BB 안전 종료는 모든 Region보다 우선하는 전역 판단입니다. pending 주문이 있으면 취소·조화·조정을 먼저 하고, 포지션이 있으면 청산 결과를 기다리는 **STOPPING**으로 이동합니다. 이때 모든 신호 영역이 먼저 자연스럽게 Final에 도달할 필요는 없습니다. 상태 구성은 **TRADE_MANAGEMENT** 밖에서 하위 Region이 활성화되어 있는 조합을 거부합니다.

11. 확인한 범위와 구현 해석 시 주의점

교수님, 이번 설명은 설계 문서와 현재 함수 본문을 대조하고, 핵심 실행 계약을 확인하는 기존 테스트를 실행한 결과입니다. 추천부의 STM·Guard·Controller, 거래 STM·이벤트 큐, 백그라운드 worker, 두 실행 흐름의 통합 테스트를 합쳐 **72개가 통과**했습니다. 특히 세 영역 초기 진입, 체결 이벤트의 세 영역 전달, 동일 snapshot의 C 매수 우선, 내부 이벤트 우선 처리, 재귀 호출 차단, Context version 불일치 시 Action 차단을 검사하는 테스트가 포함되어 있습니다.

이는 122개 행의 모든 조합을 완전 탐색하거나 실거래를 검증했다는 의미는 아닙니다. 이번에는 네트워크 주문을 실행하지 않았으며, 부록의 ID 대조도 실행 효과의 전체 증명은 아닙니다.

설계와 코드를 설명할 때는 다음 구분을 유지하겠습니다.

- Region 분리는 별도 상태 필드와 전이 함수로 실제 구현되어 있습니다. 실행 방식은 직렬 이벤트 처리에 기반한 논리적 동시성입니다.
- 추천은 TYPE_0~4를 모두 계산하지만 현재 매매 registry는 TYPE_0만 지원합니다.
- 도식의 **LOGIC_ENABLED**는 별도의 중첩 객체로 구현하지 않고 **RootState**와 전역 전이로 표현합니다. 실제 열거형에는 시작 전 **NOT_STARTED**도 있습니다.
- 표의 ID와 함수가 항상 일대일인 것은 아닙니다. 초기 전이는 상위 전이에 함께 기록하고, 공통 주문·조건·재시도는 helper와 Controller에 모았습니다.
- Context version 충돌 시 외부 Action 실행 전에 STM 결과를 되돌리는 장치가 있습니다. 이것을 이미 실행된 거래소 주문까지 되돌릴 수 있는 데이터베이스 트랜잭션으로 해석해서는 안 됩니다.
- 코드에는 표를 보완하는 실행 경로도 있습니다. 예를 들어 **BUY_RISK_BLOCKED**는 소유권 전이 파일에서 O-03/O-07 ID를 사용해 신호 상태를 돌려놓습니다. ID가 같다고 이벤트 종류까지 표의 한 행과 언제나 동일한 것은 아닙니다.

발표에서는 “세 Region이 각각 자신의 진행 상태를 기억하고 같은 사건에 함께 반응하도록 구성했습니다. 실제 계산과 주문은 정해진 순서로 처리하여 C 우선권과 단일 포지션을 지킵니다”라고 설명하겠습니다.

12. TradingController와 TradingSTM의 메시지 전달 및 Action 실행 구조

교수님, 이 장에서는 “이벤트가 발생한다”라는 표현을 실제 호출 관계로 풀어 설명하겠습니다. 이 프로젝트에서 Controller가 STM에 보내는 메시지는 ****TradingEvent** 객체와 그 판단에 사용할 **TradingContextView**입니다. STM이 돌려주는 응답은 ****TradingSTMResult**이고, Controller가 수행할 작업은 그 안의 ****순서가 있는 action_requests**에 담깁니다.

이 두 객체 사이의 메시지는 같은 백엔드 프로세스 안의 Python 함수 호출입니다. 시장 데이터가 들어오는 WebSocket이나 주문을 전송하는 REST 통신은 외부 Gateway 쪽에서 일어납니다. STM에는 거래소의 원시 메시지를 바로 보내지 않습니다.

12.1 누가 이벤트를 만들고, 누가 전달하는가

이벤트의 원인이 생기는 곳, 프로그램 내부 이벤트 객체를 만드는 곳, STM에 전달하는 곳은 구분해서 읽어야 합니다.

사건의 종류	내부 이벤트 생성·해석 주체	STM에 도달하는 경로
가격·봉 데이터 갱신	MarketDataController가 지표 평가를 준비하고, TradingController의 <code>observe_market_evaluation()</code> 이 <code>TradingEvent</code> 를 만듭니다.	<code>enqueue_event()</code> → 직렬 큐 → event processor → <code>TradingSTM.handle()</code>
현재 Region 의 조건 재검사	STM의 <code>_resolve*_region_event()</code> 가 시장 갱신 이벤트를 해당 Region의 검사 이벤트로 바꿔 해석합니다.	같은 <code>handle()</code> 호출 안에서 각 전이 함수에 전달합니다. 새 큐 항목을 추가하는 동작은 아닙니다.
즉시 이 어야 할 내부 사 건	STM은 <code>QueueEvent</code> 요청을 반환합니다. event processor가 실제 <code>TradingEvent</code> 를 만들어 넣습니다.	현재 Action 묶음 종료 → 내부 큐 → 다음 microstep의 <code>handle()</code>
시간이 지나거나 시장·봉 조건이 바뀐 뒤 재평가	STM은 <code>ScheduleReevaluation</code> 을 요청하고, Controller가 등록합니다. <code>_EventDrivenScheduler.release()</code> 가 전달받은 trigger와 due 시각을 검사해 이벤트 객체를 만듭니다.	Controller의 <code>trigger_scheduled_evaluations()</code> → 큐 → <code>handle()</code>
매수·매 도 결과	Gateway가 <code>OrderResult</code> 로 정규화합니다. Controller가 체결·Position·저장을 조정하고 <code>_create_order_outcome_event()</code> 로 전략 결과 이벤트를 만듭니다.	동기 Action 반환값 또는 <code>_enqueue_order_outcomes()</code> → 내부 큐 → <code>TradingSTM.order_finished()</code> → <code>handle()</code>
사용자 시작 명 령	<code>start_trading()</code> 이 STM의 <code>run()</code> 을 호출하고, <code>run()</code> 내부에서 <code>LOGIC_STARTED</code> 를 만듭니다.	<code>run(context)</code> → <code>handle()</code> → Controller의 <code>_apply_stm_result()</code>
사용자 중지 명 령	<code>stop_trading()</code> 이 <code>STOP_CONFIRMED</code> 를 만듭니다.	세션 잠금 안에서 <code>handle(event, context)</code> 직접 호출 → <code>_apply_stm_result()</code>

시장 평가의 연결은 `_publish_market_evaluation`, 실제 이벤트 생성은 `observe_market_evaluation`에서 확인할 수 있습니다. 여기에서 `event_id`, 발생 시각, 시장 version, 원본 평가 snapshot을 함께 보존합니다. 큐에서 기다리는 동안 더 새로운 가격이 도착하더라도 어떤 입력에서 생긴 사건인지 잃지 않기 위해서입니다.

`observe_market_evaluation()`이 반환하는 값은 큐에 수락된 이벤트입니다. 이 함수가 바로 STM 판단이나 거래소 주문 성공을 반환하는 것은 아닙니다. STM 응답은 이후 event processor가 해당 이벤트를 꺼내 처리할 때 만들어집니다.

시작·중지는 일반 시장 이벤트와 다른 직접 호출 경로를 가집니다. 따라서 “모든 메시지는 반드시 큐를 통과한다”라고 설명하면 현재 코드와 다릅니다. 직접 호출도 세션 잠금으로 처리 순서를 보호하고, Action 실행에서 생긴 주문 결과는 큐로 넘겨 다음 microstep에서 처리합니다. 구체적인 경계는 `start_trading`, `stop_trading`, `_apply_stm_result`입니다.

12.2 Controller가 STM을 호출하도록 연결하는 방법

교수님, 일반 이벤트 경로에서는 Controller가 **RunToCompletionEventProcessor**라는 전달 담당 객체를 구성합니다. 이 객체에 STM, Context 제공 함수, Action 실행 함수를 넣어 줍니다.

출처: Controller가 전달·실행 담당을 연결하는 부분

```
self._event_queue = SerialEventQueue()
self._event_processor = RunToCompletionEventProcessor(
    stm=selected_stm,
    context_provider=self._context.snapshot,
    action_executor=self._execute_action,
    event_queue=self._event_queue,
    clock=self._clock,
    order_finished_observer=self._record_order_finished_trace,
    event_processing_observer=self._observe_processing_event,
    event_context_preparer=self._prepare_market_event_context,
    result_observer=self._record_indicator_evaluation,
)
```

이 연결에서 각 인자는 다음 책임을 가집니다.

- **stm=selected_stm**: 판단을 요청할 TradingSTM 인스턴스입니다.
- **context_provider=self._context.snapshot**: 판단 시점에 읽기 전용 Context를 얻는 함수입니다.
- **event_context_preparer=self._prepare_market_event_context**: 큐에서 꺼낸 원본 시장 평가와 현재 runtime을 맞춰 준비하는 함수입니다.
- **action_executor=self._execute_action**: STM 응답에 있는 Action을 실제 수행할 **Controller의 함수**입니다.
- **event_queue=self._event_queue**: 이벤트를 한 개씩 꺼낼 세션 전용 큐입니다.

따라서 “Controller가 STM에 메시지를 보내고 응답대로 행동한다”는 책임은 유지됩니다. 호출과 응답 반복을 event processor에 맡기고, 실제 행동은 Controller의 callback으로 돌려받는 구성입니다. STM은 이 callback을 직접 호출하지 않으며 Controller 참조를 알 필요도 없습니다.

큐에 새 일이 들어오면 **enqueue_event**가 실행 작업자에게 처리를 요청합니다. **_TradingEventRuntimeWorker**가 Controller의 **run_event_runtime_cycle()**을 구동하고, 이 함수는 예약된 재시도를 해제한 뒤 **drain_events()**를 통해 큐를 처리합니다. 일반 실행에서 매번 UI가 “다음 이벤트를 처리하라”고 호출할 필요는 없습니다.

12.3 전달하는 메시지와 돌려받는 응답의 의미

events.py의 **TradingEvent**는 사건 이름과 식별·출처 정보를 담습니다. 실제 Guard는 이 이벤트와 **context.py**의 **TradingContextView**를 함께 사용합니다. 예를 들어 **MARKET_DATA_UPDATED**라는 이름만으로 매수를 판단할 수는 없고, %B·경과시간·pending 주문·현재 소유자까지 알아야 합니다.

process_next는 큐에서 이벤트를 꺼내 원본 평가를 Context에 준비한 뒤 snapshot을 얻습니다. 이후 주문 결과인지에 따라 아래처럼 호출합니다.

출처: 이벤트 처리기가 STM의 응답을 받는 부분

```

is_order_finished_event = event.event_type in _ORDER_FINISHED_EVENT_TYPES
if is_order_finished_event:
    result = self._stm.order_finished(
        event,
        context,
    ) # Position과 durable history가 반영된 최신 Context만 전달한다.
else:
    result = self._stm.handle(
        event,
        context,
    ) # 시장·timer·사용자 event는 기존 일반 경계를 유지한다.

```

`order_finished()`도 그 자체로 주문을 실행하는 함수는 아닙니다. 허용된 주문 결과 이벤트인지 확인한 뒤 `handle()`로 넘기는 STM의 입력 경계입니다. 주문과 체결 반영은 이미 Controller 쪽에서 처리한 상태입니다.

STM은 여러 Region을 평가하고 다음 내용을 가진 `TradingSTMResult`를 반환합니다.

응답 필드	Controller가 해석하는 의미
<code>decision_id</code>	어느 이벤트·Context version·전이 조합에서 나온 판단인지 추적하는 ID
<code>consumed</code>	현재 상태에서 처리할 전이를 선택했는지 여부. 주문 체결 성공 여부가 아닙니다.
<code>transition_ids</code>	선택된 표의 ID들. 여러 Region이 반응하면 한 응답에 여러 ID가 들어갑니다.
<code>state_before / state_after</code>	전체 Region 구성의 판단 전·후 상태
<code>action_requests</code>	Controller가 순서대로 실행해야 하는 요청 객체들의 tuple
<code>context_version</code>	이 판단에 사용한 Context의 version

전이가 없으면 STM은 `consumed=False`, 빈 전이 ID와 빈 Action 목록을 반환하고 상태를 유지합니다. 전이가 있더라도 Action이 없는 행이면 Controller가 수행할 외부 작업은 없습니다.

12.4 응답을 받은 뒤 표의 Action을 행동으로 바꾸는 방법

event processor는 `result.action_requests`를 앞에서부터 순회하면서 연결해 둔 `self._action_executor(action)`을 호출합니다. 이 호출의 실제 대상이 `TradingController._execute_action()`입니다. Controller는 ID 문자열을 보고 표를 다시 검색하는 대신, **Action 객체의 자료형과 payload**를 보고 수행할 함수를 선택합니다.

예를 들어 `PatchRuntimeContext`는 Context 변경으로, `SubmitOrder`는 주문 실행 경로로, `ScheduleReevaluation`은 scheduler 등록으로 분배합니다. `QueueEvent`는 Controller에서 실행 요청을 기록하고 processor가 실제 큐 삽입을 말합니다. 공통 실행 분기는 `_execute_action`에서 볼 수 있습니다.

Action 전에 Context version도 확인합니다. STM이 판단한 뒤 입력이 바뀌었다면 processor는 아직 외부 효과를 실행하지 않은 STM 결과를 되돌리고, 이벤트를 큐에 복원합니다. 일치할 때만 요청들을 실행합니다. 이는 `process_next`의 `context_version` 비교 부분입니다.

한 Action의 실행 결과가 새로운 `TradingEvent`를 반환할 수도 있습니다. processor는 이를 모아 두었다가 Action 묶음이 끝난 뒤 내부 우선순위로 큐에 넣습니다. 따라서 아직 B/C의 runtime 변경이 끝나지 않았는데 그 중간에 체결 이벤트를 재

귀 처리하는 흐름을 만들지 않습니다.

메시지의 한 왕복을 표현하면 다음과 같습니다.

```
시장 평가에서 TradingEvent 생성
→ Controller의 세션 큐에 등록
→ processor가 이벤트와 Context를 STM에 전달
→ STM이 전이 ID·다음 상태·Action 목록 반환
→ processor가 Controller의 Action 실행 함수를 순서대로 호출
→ Context 변경·주문·저장·예약 수행
→ 필요한 결과 이벤트를 큐에 등록
→ 다음 microstep에서 STM이 결과를 다시 판단
```

12.5 주문 응답이 나중에 도착하는 경우

거래소가 최초 응답에서 체결을 확정할 수도 있고, 이후 WebSocket이나 동일 주문 조회에서 결과가 확인될 수도 있습니다. 초기 제출 응답은 `_submit_order_action`에서 `_handle_order_result()`로 전달합니다. 뒤늦은 주문 스트림 결과는 `apply_order_stream_result`가 `observe_order_result`로 전달하고, 같은 주문 상태를 찾아 같은 결과 처리 경로에 합류 시킵니다.

`OrderResult`는 거래소 주문의 관찰 결과이고, `CASE_C_POSITION_OPENED`는 그 결과를 실제 포지션과 이력에 반영한 뒤 전략 판단에 알려 주는 사건입니다. 따라서 거래소에서 `FILLED`를 받았다는 사실만 보고 STM의 보유 상태를 먼저 바꾸지 않습니다.

상태 불명·부분 체결은 같은 주문 ID의 조회·조정 경로를 거칩니다. 체결 수량이 없는 terminal 실패라면 실패 이벤트를 만들고, STM이 재시도를 요청하면 Controller가 backoff를 적용해 예약합니다. 이와 관련된 함수는 `_handle_order_result()`, `_finalize_terminal_execution()`, `_complete_terminal_after_history()`, `_enqueue_order_outcomes()`입니다.

12.6 실제 코드로 따라가는 예시: C 매수 요청부터 세 Region의 체결 처리까지

이제 6·7장에서 설명한 C 매수 사례를 실제 코드와 연결하겠습니다. 아래 Python 블록은 현재 소스의 발췌이며, 들여쓰기만 읽기 편하게 정리했습니다. 전체 함수 중 설명 대상 분기를 보여 주므로 독립 실행용 예제는 아닙니다. 생략한 검증·예외 경로는 각 출처 링크에서 확인할 수 있습니다.

설명 입력은 B가 `B_WAIT_PULLBACK`, C가 `C_SETUP`, 실제 포지션과 pending 주문은 없는 상태입니다. 이전 flush 가격은 89, 현재가는 90, 현재 %B와 C 진입선은 -0.18, C 회복 경과는 2분, B 신호 경과는 1시간이라고 하겠습니다. 같은 봉·같은 하단 이벤트 안에 있으며 상단 종료나 새 터치 재초기화는 발생하지 않는 상황입니다. 실제 주문 부분은 주문 실행·위험 검사·저장이 모두 허용되고 성공하는 경우를 가정합니다. 이 설명을 위해 실거래를 실행한 것은 아닙니다.

예시 ① `TradingController`가 시장 이벤트 객체를 만듭니다.

출처: 시장 평가를 `TradingEvent`로 만드는 부분

```
evaluation_time = self._clock()
event_type = self._select_market_event_type(market)
event = TradingEvent(
    event_type=event_type,
    occurred_at=evaluation_time,
```

```

        priority=EventPriority.MARKET,
        event_id=f"market:{market_version}:{source_event_id}",
        lower_event_id=self._context.runtime.lower_event_id,
        candle_id=market.current_30m_candle_id,
        market_evaluation=market,
        market_version=market_version,
    )
    return self.enqueue_event(event) # Context mutation은 FIFO claim 직후 준비 단
    계에서만 수행한다.

```

여기서 `market_evaluation=market`이 원본 평가 입력을 이벤트 안에 붙입니다. 예시 상황은 현재 하단 이벤트 안의 일반 가격 갱신이므로 `MARKET_DATA_UPDATED`로 처리됩니다. `enqueue_event()`는 큐에 넣고 작업자를 깨웁니다. 큐에서 꺼낼 때 `_prepare_market_event_context()`가 처리 시점 `runtime`을 반영해 이벤트 종류를 다시 분류하고 Context를 준비합니다.

예시 ② STM이 같은 시장 이벤트를 C setup 검사로 해석합니다.

출처: C Region용 이벤트 해석

```

if event.event_type is not TradingEventType.MARKET_DATA_UPDATED:
    return event

# 현재 활성 신호 상태에서 수신 가능한 재평가 이벤트만 생성한다.
if state.case_c_signal_state is CaseCSignalState.C_WAIT_SETUP:
    return replace(event, event_type=TradingEventType.RETRY_C_WAIT_SETUP)
if state.case_c_signal_state is CaseCSignalState.C_SETUP:
    return replace(
        event,
        event_type=TradingEventType.RETRY_CASE_C_SETUP_CONDITION_CHECK,
    )

return event

```

C가 `C_SETUP`이므로 C Region에는 `RETRY_CASE_C_SETUP_CONDITION_CHECK`로 전달합니다. `replace()`는 같은 사건의 평가 입력과 식별 정보는 보존하면서 이벤트 종류를 바꾼 객체를 만듭니다. 여기서 별도 큐 항목을 생성하거나 매수를 수행하는 것은 아닙니다. B Region에는 자신의 상태에 맞는 놀림목 검사 이벤트가 전달됩니다.

예시 ③ C-12의 Guard가 만족되어 매수 요청과 다음 상태를 만듭니다.

출처: C-12의 조건·다음 상태·Action 구성

```

entry_pct_b = runtime.entry_pct_b
in_recovery_window = (
    runtime.flush_low is not None
    and entry_pct_b is not None
    and condition_met("c_recovery_window", context)
    and condition_met("c_rebound", context)
    and condition_unmet("c_recovery", context)
)

```

```
# owner와 pending 주문이 모두 없을 때만 Case C 최초 매수를 요청한다.
if (
    runtime.position_owner is None
    and runtime.pending_order_id is None
    and in_recovery_window
    and condition_met("c_entry_limit", context)
):
    actions = (
        patch(current_open_pct_b=market.realtime_pct_b),
        *create_entry_order_actions(
            StrategyType.CASE_C,
            OrderAttemptKind.INITIAL,
            event,
            context,
        ),
    )
    return create_transition_outcome(
        "C-12",
        replace(
            state,
            case_c_signal_state=CaseCSignalState.C_POSITION_OPEN_SIGNALLED,
        ),
        *actions,
    )
```

예시에서는 $2\text{분} \leq 3\text{분}$, 현재 $\%B \geq$ 진입선, 현재 $\%B < 0.25$, 진입선 < -0.15 가 성립하고 소유자·pending 주문도 없습니다. 따라서 C-12가 선택됩니다. `replace(state, ...)`는 **C 신호 상태만 C_POSITION_OPEN_SIGNALLED로 바꾼 후보**를 만듭니다. 아직 실제 포지션 소유자를 C로 설정하지 않습니다.

이때 `create_entry_order_actions()`가 만드는 실제 요청 내용은 다음과 같습니다.

출처: 매수 요청을 두 Action으로 만드는 helper

```
intent_id = (
    context.runtime.pending_intent_id
    or f"{strategy}:BUY:{context.runtime.lower_event_id}:"
    f"{event.sequence_number}"
)
return (
    patch(
        pending_strategy=strategy,
        pending_order_side=OrderSide.BUY,
        pending_order_attempt_kind=attempt_kind,
        pending_intent_id=intent_id,
        trading_phase=TradingPhase.ENTRY_ORDER_PENDING,
    ),
    SubmitOrder(
        strategy=strategy,
        side=OrderSide.BUY,
        attempt_kind=attempt_kind,
        idempotency_key=intent_id,
```

```
    ),
)
```

C-12 응답의 Action 순서는 **현재 %B 기록 → 주문 의도와 pending phase 예약 → C 매수 제출 요청**입니다.

`patch(...)`와 `SubmitOrder(...)`는 데이터를 담은 요청 객체를 생성할 뿐, 이 helper 안에서 Context나 거래소를 변경하지 않습니다.

같은 microstep에서 B도 매수 가능하다면 `_does_case_c_buy_win()`이 B 후보를 제외합니다. 따라서 이 예시의 최종 전이 ID는 ('C-12',)이고 B는 `B_WAIT_PULLBACK`을 유지합니다.

예시 ㉔ STM이 판단 결과를 응답으로 반환합니다.

출처: [TradingSTMResult를 반환하는 부분](#)

```
self._state = selected.state_after
return TradingSTMResult(
    decision_id=create_decision_id(event, context,
selected.transition_ids),
    consumed=True,
    transition_ids=selected.transition_ids,
    state_before=state_before,
    state_after=selected.state_after,
    action_requests=selected.action_requests,
    context_version=context.version,
)
```

이 코드는 STM 자신의 상태를 확정하고 `TradingSTMResult`를 반환합니다. Controller가 실행할 요청들은 `selected.action_requests`에 담겨 있습니다. 여기까지의 응답 의미는 “C 매수를 요청하겠습니다”이지 “C 매수가 체결되었습니다”가 아닙니다.

예시 ㉕ event processor가 Controller의 실행 함수를 불러 실제로 행동합니다.

processor의 Action 반복문에서 일반 Action을 실행하는 부분은 아래와 같습니다. `QueueEvent`는 바로 앞의 별도 분기에서 후속 삽입용으로 모으고, 그 외 Action은 이 코드로 처리합니다.

출처: [Controller callback을 호출하고 결과 이벤트를 모으는 부분](#)

```
action_result = self._action_executor(action)
if inspect.isawaitable(action_result):
    action_result = await action_result
if action_result is not None:
    returned_events.extend(action_result)
```

`self._action_executor`는 세션 생성 때 주입한 Controller의 `_execute_action`입니다. 따라서 이 한 줄이 STM의 반환값에 담긴 요청을 Controller의 실제 행동으로 연결합니다.

processor가 요청 목록을 순회하면서 호출하는 대상은 앞서 연결한 `self._execute_action`입니다. Context 변경 요청은 다음 분기로 들어갑니다.

출처: Context 변경 Action 실행

```
if isinstance(action, PatchRuntimeContext):
    trace_identity = self._prepare_order_patch_trace(action)
    self._context.apply_runtime_patch(action)
    self._complete_order_patch_trace(trace_identity)
    return ()
```

먼저 %B와 pending 관련 값들이 실제 Context에 적용됩니다. 이어서 **SubmitOrder**는 아래 분기로 들어갑니다.

출처: SubmitOrder 실행 분기

```
if isinstance(action, SubmitOrder):
    return self._submit_order_action(action)
```

`_submit_order_action()`은 주문 실행 조건, 수량·위험 검사, 주문 식별자와 복구 기록을 준비하고, Gateway 호출 전에 pending 주문 ID를 게시합니다. 정상적인 일반 주문의 외부 호출은 `self._api_gateway.submit_order(order)`이고, 결과는 `_handle_order_result(state, result, initial=True)`로 전달합니다. 실제 제출 지점은 `_submit_order_action`에서 확인할 수 있습니다. STM은 이 Gateway를 호출하지 않습니다.

예시 ⑥ 실제 체결을 반영한 뒤 새로운 전략 이벤트를 만듭니다.

체결을 Position에 반영한 뒤 `_publish_position_to_context`가 실제 수량과 owner를 Context에 적용합니다. 저장과 terminal 정리가 끝나면 `_complete_terminal_after_history()`가 `_create_order_outcome_event()`를 호출합니다. 성공한 C 매수에 사용할 이벤트 이름은 다음 코드에서 선택합니다.

출처: 체결 결과를 전략 이벤트 이름으로 바꾸는 부분

```
if strategy is StrategyType.CASE_B:
    return (
        TradingEventType.CASE_B_POSITION_OPENED
        if succeeded
        else TradingEventType.CASE_B_BUY_FAILED
    )
return (
    TradingEventType.CASE_C_POSITION_OPENED
    if succeeded
    else TradingEventType.CASE_C_BUY_FAILED
)
```

예시의 `strategy`는 `CASE_C`이고 `succeeded=True`이므로 `CASE_C_POSITION_OPENED`가 됩니다. `_create_order_outcome_event()`는 이 이름에 주문 ID, 안정적인 이벤트 ID, 발생 시각과 payload를 붙여 `TradingEvent`를 만듭니다.

주문 제출 Action 안에서 결과가 확정되어 반환되었다면 processor가 아래 코드로 큐에 넣습니다.

출처: [Action이 반환한 주문 결과를 다음 microstep에 전달](#)

```
for returned_event in returned_events:
    self._queue.enqueue(returned_event, internal=True)
```

뒤늦은 WebSocket·조회 응답에서 확정되었다면 Controller의 `_enqueue_order_outcomes()`가 같은 방식으로 내부 우선순위를 적용합니다. 두 경로 모두 STM을 곧바로 재귀 호출하지 않고 다음 microstep으로 넘깁니다.

예시 ㉔ 동일 체결 이벤트가 **Region 1·3·2**에 전달됩니다.

다음 큐 처리에서 processor는 이 이벤트를 `stm.order_finished(event, context)`에 전달합니다. 이 시점의 Context에는 실제 포지션과 C 소유자, pending 정리가 반영되어 있습니다. Region 1의 소유권 코드는 이를 확인합니다.

출처: [C 체결 후 Region 1을 여는 부분](#)

```
if (
    event_type is TradingEventType.CASE_C_POSITION_OPENED
    and runtime.position_owner is StrategyType.CASE_C
    and runtime.pending_order_id is None
    and context.position.is_open
):
    state_after = replace(
        state,
        ownership_state=OwnershipState.CASE_C_POSITION_MANAGEMENT,
        case_c_position_state=CaseCPositionState.CASE_C_HOLDING,
    )
    return create_transition_outcome(
        "0-06",
        state_after,
        patch(case_b_entry_paused=True),
    )

    create_queue_event_action(TradingEventType.START_CASE_C_CONDITION_CHECK,
    context),
    extra_transition_ids=("PC-01",),
)
```

여기에서 0-06과 PC-01이 함께 기록되고 Region 1은 CASE_C_HOLDING을 활성화합니다. 동시에 `patch(case_b_entry_paused=True)`와 `START_CASE_C_CONDITION_CHECK`의 후속 요청을 만듭니다. 같은 원래 체결 이벤트는 이어서 C 신호 Region의 C-16과 B 신호 Region의 B-13에도 전달됩니다.

이 동작은 기존 `test_position_open_feedback_is_broadcast_to_three_regions` 테스트가 다음처럼 확인합니다.

출처: [여러 Region에 대한 실제 테스트 assertion](#)

```
self.assertEqual(("0-06", "PC-01", "C-16", "B-13"), result.transition_ids)
```

따라서 **O-06 → PC-01 → C-16 → B-13**은 네 번의 별도 이벤트 왕복이 아니라, 하나의 체결 이벤트를 처리한 응답에 함께 담기는 전이 ID 순서입니다. 이 응답의 Action들을 적용한 뒤에 **START_CASE_C_CONDITION_CHECK**라는 별도 이벤트가 다음 microstep에서 C 보유 조건을 검사합니다.

이 예시에서 실제 호출 경계를 정리하면 **TradingController가 사건 객체 생성 → processor가 STM 호출 → STM이 C-12 요청 반환 → Controller가 매수 수행 → Controller가 체결 사건 생성 → processor가 STM 재호출 → 여러 Region의 전이와 후속 Action 실행**이 됩니다. 소스에서 메시지가 오가는 지점과 표의 Action이 실제 행동으로 바뀌는 지점을 이 순서대로 따라가실 수 있습니다.

부록. 두 표의 모든 ID와 실제 코드 위치

아래 링크는 ID가 **실제 분기·전이 선언·초기 전이 묶음에 쓰인 위치**를 가리킵니다. Trading의 단순 ID catalog는 제외했습니다. 한 행에 링크가 여러 개 있으면 초기 진입을 여러 경로에서 재사용하거나 같은 ID를 보완 경로에서도 사용한 경우입니다. 상수 매핑에 있는 ID는 그 매핑을 사용하는 같은 파일의 처리 함수까지 함께 읽으면 됩니다. 이벤트 열은 원본 표의 표현을 유지했습니다. 줄 번호는 작성 시점 기준입니다.

4H_REGIME_Event_Action_Table.md — 13개

ID	표의 이벤트	실제 소스 위치
EA-001	INITIAL_EVALUATION_REQUESTED	transitions.py:143 · transitions.py:244
EA-002	EVALUATION_READY	transitions.py:150 · transitions.py:245
EA-003	EVALUATION_READY	transitions.py:158 · transitions.py:246
EA-004	EVALUATION_READY	transitions.py:166 · transitions.py:247
EA-005	EVALUATION_READY	transitions.py:174 · transitions.py:248
EA-006	EVALUATION_READY	transitions.py:182 · transitions.py:249
EA-007	EVALUATION_READY	transitions.py:190 · transitions.py:250
EA-008	EVALUATION_READY	transitions.py:198 · transitions.py:251
EA-101	FOUR_HOUR_CANDLE_CLOSED	transitions.py:206 · transitions.py:252
EA-102	FOUR_HOUR_CANDLE_CLOSED	transitions.py:213 · transitions.py:253
EA-103	FOUR_HOUR_CANDLE_CLOSED	transitions.py:220 · transitions.py:254
EA-104	FOUR_HOUR_CANDLE_CLOSED	transitions.py:227 · transitions.py:255
EA-105	FOUR_HOUR_CANDLE_CLOSED	transitions.py:234 · transitions.py:256

Trading_Logic_Event_Action_Table.md — 109개

ID	표의 이벤트	실제 소스 위치
G-01	로직 시작	global_transitions.py:248
G-02	하단 BB 접촉	global_transitions.py:275

ID	표의 이벤트	실제 소스 위치
G-03	새 30분봉 하단 BB 접촉	global_transitions.py:303
G-04	TRADE_MANAGEMENT 완료	global_transitions.py:325
G-05	매매 중지	global_transitions.py:169
G-06	매매 중지[포지션 보유]	global_transitions.py:159
G-06P	매매 중지[pending 주문 존재]	global_transitions.py:150
G-06F	FORCE_SELL_FINISHED	global_transitions.py:197
G-06R	FORCE_SELL_FAILED	global_transitions.py:228
G-07	UPPER_BAND_TOUCHED	global_transitions.py:59 · global_transitions.py:76 · global_transitions.py:86
O-01	초기 진입	case_b_position_transitions.py:129 · global_transitions.py:284 · global_transitions.py:313
O-02	CASE_B_POSITION_OPENED	ownership_transitions.py:105
O-03	CASE_B_BUY_FAILED	ownership_transitions.py:70 · ownership_transitions.py:118
O-04	CASE_B_BUY_RETRY	ownership_transitions.py:142
O-05	CASE_B_BUY_FAILED	ownership_transitions.py:150
O-06	CASE_C_POSITION_OPENED	ownership_transitions.py:172
O-07	CASE_C_BUY_FAILED	ownership_transitions.py:83 · ownership_transitions.py:186
O-08	CASE_C_BUY_RETRY	ownership_transitions.py:209
O-09	CASE_C_BUY_FAILED	ownership_transitions.py:217

ID	표의 이벤트	실제 소스 위치
PB-01	초기 진입	ownership_transitions.py:108
PB-02	START_CASE_B_CONDITION_CHECK	case_b_position_transitions.py:190
PB-03	START_CASE_B_CONDITION_CHECK	case_b_position_transitions.py:199
PB-04	RETRY_CASE_B_CONDITION_CHECK	case_b_position_transitions.py:190
PB-05	RETRY_CASE_B_CONDITION_CHECK	case_b_position_transitions.py:199
PB-06	CASE_B_EMERGENCY_STOP(비상 손절)	case_b_position_transitions.py:48
PB-07	CASE_B_SELL_FAILED	case_b_position_transitions.py:57
PB-08	CASE_B_STOP(일반 손절)	case_b_position_transitions.py:51
PB-09	CASE_B_SELL_FAILED	case_b_position_transitions.py:58
PB-10	CASE_B_TIME_EXIT(시간 청산)	case_b_position_transitions.py:52
PB-11	CASE_B_SELL_FAILED	case_b_position_transitions.py:59
PB-12	CASE_B_TAKE_PROFIT(일반 익절)	case_b_position_transitions.py:53
PB-13	CASE_B_SELL_FAILED	case_b_position_transitions.py:60
PB-14	CASE_B_UPPER_TREND(강한 반등)	case_b_position_transitions.py:225
PB-15	CASE_B_TREND_HOLD_CONDITION_CHECK	case_b_position_transitions.py:307
PB-16	CASE_B_TREND_HOLD_CONDITION_CHECK	case_b_position_transitions.py:316
PB-17	CASE_B_TREND_HOLD_SELL	case_b_position_transitions.py:334
PB-18	CASE_B_SELL_FAILED	case_b_position_transitions.py:342

ID	표의 이벤트	실제 소스 위치
PB-19	CASE_B_SELL_RETRY	case_b_position_transitions.py:281
PB-20	CASE_B_SELL_FAILED	case_b_position_transitions.py:247
PB-21	CASE_B_SELL_RETRY	case_b_position_transitions.py:371
PB-22	CASE_B_SELL_FAILED	case_b_position_transitions.py:344
PB-23F	CASE_B_SELL_FILLED	case_b_position_transitions.py:97
PB-23	CASE_B_SELL_FINISHED	case_b_position_transitions.py:117
PB-24	CASE_B_SELL_FINISHED	case_b_position_transitions.py:133
PC-01	초기 진입	ownership_transitions.py:176
PC-02	START_CASE_C_CONDITION_CHECK	case_c_position_transitions.py:185
PC-03	START_CASE_C_CONDITION_CHECK	case_c_position_transitions.py:194
PC-04	RETRY_CASE_C_CONDITION_CHECK	case_c_position_transitions.py:185
PC-05	RETRY_CASE_C_CONDITION_CHECK	case_c_position_transitions.py:194
PC-06	CASE_C_STOP(손절)	case_c_position_transitions.py:213
PC-07	CASE_C_SELL_FAILED	case_c_position_transitions.py:255
PC-08	CASE_C_TIME_EXIT(시간 청산)	case_c_position_transitions.py:449
PC-09	CASE_C_SELL_FAILED	case_c_position_transitions.py:456
PC-10	CASE_C_ENTER_PROFIT_ZONE(익절권 진입)	case_c_position_transitions.py:226
PC-11	START_TP_TRAILING_CONDITION_CHECK	case_c_position_transitions.py:307

ID	표의 이벤트	실제 소스 위치
PC-12	START_TP_TRAILING_CONDITION_CHECK	case_c_position_transitions.py:317
PC-13	RETRY_TP_TRAILING_CONDITION_CHECK	case_c_position_transitions.py:307
PC-14	RETRY_TP_TRAILING_CONDITION_CHECK	case_c_position_transitions.py:317
PC-15	CASE_C_EMA_INCREASEMENT(30m EMA slope 증가)	case_c_position_transitions.py:330
PC-16	CASE_C_SELL_AT_TP_PRICE(회복 강도 약화)	case_c_position_transitions.py:354
PC-17	CASE_C_SELL_FAILED	case_c_position_transitions.py:392
PC-18	CASE_C_EMA_DECREASEMENT(30m EMA slope 비증가)	case_c_position_transitions.py:372
PC-19	CASE_C_SELL_FAILED	case_c_position_transitions.py:394
PC-20	CASE_C_SELL_RETRY	case_c_position_transitions.py:276
PC-21	CASE_C_SELL_FAILED	case_c_position_transitions.py:244 · case_c_position_transitions.py:257
PC-22	CASE_C_SELL_RETRY	case_c_position_transitions.py:411
PC-23	CASE_C_SELL_FAILED	case_c_position_transitions.py:379 · case_c_position_transitions.py:390
PC-23F	CASE_C_SELL_FILLED	case_c_position_transitions.py:64
PC-24	CASE_C_SELL_FINISHED	case_c_position_transitions.py:75
PC-25	CHECK_CASE_C_RECOVERY	case_c_position_transitions.py:88
PC-26	CHECK_CASE_C_RECOVERY	case_c_position_transitions.py:99
PC-27	CHECK_CASE_B_HANDOFF	case_c_position_transitions.py:121

ID	표의 이벤트	실제 소스 위치
PC-28	CHECK_CASE_B_HANDOFF	case_c_position_transitions.py:131
B-01	초기 진입	case_b_position_transitions.py:129 · global_transitions.py:284 · global_transitions.py:313
B-02	ACTIVATE_TRADE_MANAGEMENT	case_b_signal_transitions.py:62
B-03	ACTIVATE_TRADE_MANAGEMENT	case_b_signal_transitions.py:68
B-04	30M_CANDLE_CLOSED	case_b_signal_transitions.py:85
B-05	30M_CANDLE_CLOSED	case_b_signal_transitions.py:97
B-06	START_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	case_b_signal_transitions.py:164
B-07	START_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	case_b_signal_transitions.py:182
B-08	START_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	case_b_signal_transitions.py:141
B-09	RETRY_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	case_b_signal_transitions.py:164
B-10	RETRY_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	case_b_signal_transitions.py:182
B-11	RETRY_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	case_b_signal_transitions.py:141
B-12	CASE_C_POSITION_OPENED	case_b_signal_transitions.py:115
B-13	CASE_C_POSITION_OPENED	case_b_signal_transitions.py:194
B-14	CASE_B_ACTIVE_RESUME	case_b_signal_transitions.py:120
B-15	CASE_B_ACTIVE_RESUME	case_b_signal_transitions.py:199
B-16	CASE_B_WAIT_ONLY	case_b_signal_transitions.py:125

ID	표의 이벤트	실제 소스 위치
B-17	CASE_B_WAIT_ONLY	case_b_signal_transitions.py:204
B-18	CASE_B_POSITION_OPENED	case_b_signal_transitions.py:214
B-19	CASE_C_POSITION_OPENED	case_b_signal_transitions.py:222
C-01	초기 진입	case_b_position_transitions.py:129 · global_transitions.py:284 · global_transitions.py:313
C-02	ACTIVATE_TRADE_MANAGEMENT	case_c_signal_transitions.py:63
C-03	ACTIVATE_TRADE_MANAGEMENT	case_c_signal_transitions.py:72
C-04	RETRY_C_WAIT_SETUP	case_c_signal_transitions.py:63
C-05	RETRY_C_WAIT_SETUP	case_c_signal_transitions.py:72
C-06	CASE_B_POSITION_OPENED	case_c_signal_transitions.py:86
C-07	START_CASE_C_SETUP_CONDITION_CHECK	case_c_signal_transitions.py:99
C-08	RETRY_CASE_C_SETUP_CONDITION_CHECK	case_c_signal_transitions.py:122
C-09	RETRY_CASE_C_SETUP_CONDITION_CHECK	case_c_signal_transitions.py:139
C-10	RETRY_CASE_C_SETUP_CONDITION_CHECK	case_c_signal_transitions.py:142
C-11	RETRY_CASE_C_SETUP_CONDITION_CHECK	case_c_signal_transitions.py:153
C-12	RETRY_CASE_C_SETUP_CONDITION_CHECK	case_c_signal_transitions.py:194
C-13	RETRY_CASE_C_SETUP_CONDITION_CHECK	case_c_signal_transitions.py:204
C-14	RETRY_CASE_C_SETUP_CONDITION_CHECK	case_c_signal_transitions.py:229

ID	표의 이벤트	실제 소스 위치
C-15	CASE_B_POSITION_OPENED	case_c_signal_transitions.py:243
C-16	CASE_C_POSITION_OPENED	case_c_signal_transitions.py:259
C-17	CASE_B_POSITION_OPENED	case_c_signal_transitions.py:267

검증에 사용한 기존 테스트

- [unit/regime/test_stm.py](#)
- [unit/regime/test_guards.py](#)
- [unit/regime/test_regime_controller.py](#)
- [unit/trading/test_stm.py](#)
- [unit/trading/test_event_queue.py](#)
- [unit/bootstrap/test_trading_event_runtime_worker.py](#)
- [integration/test_regime_evaluation_flow.py](#)
- [integration/test_trading_event_runtime_flow.py](#)